Lab 3: Threads Programming

Francesco Astolfi

Jenuary 11, 2026

1 LeNet-5

1.1 The source code

The project utilizes the professor's sequential C implementation as the performance baseline. Parallelization efforts were concentrated on the Convolutional Layers, as they represent the primary computational bottleneck in LeNet-5 architecture.
The forward propagation pipeline has been re-engineered to utilize Task-Level Parallelism through CUDA Stream Concurrency. To achieve maximum hardware utilization and overlap data transfers with kernel execution, the following optimizations were implemented:

- **Asynchronous Multi-Stream Pipeline:** The system initializes a GPUContext containing a pool of independent CUDA streams. By assigning each image in a mini-batch to a unique stream, the CPU can issue commands asynchronously, allowing the GPU's hardware scheduler to overlap the Host-to-Device (H2D) transfer of image N+1 with the Kernel Execution of image N. After each batch training there is the reuse of the data structure.
- **Encapsulated Memory Management:** For each concurrent image, the g_gpu_ctx structure maintains dedicated memory offsets for every architectural layer. This prevents race conditions during parallel forward passes and ensures that intermediate feature maps are preserved for the subsequent backward phase.
- **Latency Hiding via FIFO Queuing:** The CPU acts as a non-blocking producer, dispatching work into the FIFO hardware queues of multiple streaming channels. This architecture minimizes CPU idle time, as the processor only synchronizes with the GPU immediately prior to the gradient computation (Backward Phase) to ensure data integrity.
- **Deterministic Error Propagation:** The pipeline is designed so that operations within a single stream maintain strict causal consistency; however, independent streams execute out-of-order to saturate the GPU's Streaming Multiprocessors (SMs)
- **Constant Memory Utilization:** Small, frequently accessed filter weights (for layers C1 and C3) are stored in Constant Memory. This utilizes a specialized cache that supports high-speed broadcasting, allowing all 32 threads in a warp to read the same weight simultaneously without bank conflicts. The c5 convolution weight wasn't possible to load in the constant memory because of their big size:
$$C5 = 16 \times 120 \times 5 \times 5 \cong 385\ KB$$
While the constant memory generally hosts just a maximum of $64\ KB$ that is possible to cover with the sum of the weights of C1 and C3. For this reason, has been 2 kernel functions written one with the weights from the constant memory and the other from the main GPU memory.

Layer	Calculation (In × Out × Filter)	Total Doubles	Bytes (approx.)	Fits in 64KB?
C1	1 x 6 x 5 x 5	150	1,200 B	Yes
C3	6 x 16 x 5 x 5	2,4	19,200 B	Yes
Subtotal	C1 + C3	2,55	~20.4 KB	Yes (Remaining: ~43KB)
C5	16 x 120 x 5 x 5	48	384,000 B	NO (~384 KB)

Table 1: Constant memory weight evaluation

The weights come corrected after each backward process and then updated in the constant memory before the use in the forward process.

The convolution operation was refactored from a nested loop structure into a Tiled Convolution Kernel. This approach leverages the spatial locality of image data, where a thread block collaborates to process a specific spatial "tile" of the feature map.

Performance improvements focus on the TiledConvolutionalKernel:

- **Boundary Handling & Padding:** The kernel includes integrated boundary checks (zero-padding). If a thread's required pixel falls outside the image dimensions.
- **Redundant Memory Access Reduction:** By loading a ($tiled_size\ x\ tiled_size$) input tile into Shared Memory (SRAM), has been ensured that each input pixel is fetched from the high-latency Global Memory only once. Overlapping regions are shared among threads, drastically reducing DRAM bandwidth pressure.
- **Global Memory Coalescing:** The loading phase is implemented using a flattened indexing strategy. This ensures that threads within a warp access contiguous memory address, allowing the hardware to "coalesce" multiple requests into a single memory transaction.
- **Shared Memory Banking & Parallelism:** Each thread is mapped to a single output pixel. This 1-to-1 mapping allows the 256 threads in a block to perform the multiply-accumulate (MAC) operations in parallel, with near-zero latency for input data retrieval. Is processed one layer at time reusing also the reserve on the shared memory.
- **Thread Coarsening and Resource Reuse:** To maximize the efficiency of the Tiled Convolution Kernel, a thread coarsening strategy was implemented to exploit data reuse across input channels. Instead of assigning one thread to a single channel-specific calculation, each thread is designed to process multiple input channels for the same spatial pixel location. During the tile-loading phase, each thread performs multiple coalesced loads into Shared Memory (SRAM), populating the local cache with values from different input channels simultaneously.
- **Thread synchronization:** Implementation of __syncthreads() barriers after shared memory loading and following the convolution evaluation. This ensures data consistency within the thread block and eliminates the need for expensive global synchronization during the kernel execution.
- **ReLu activation function:** To eliminate negative values and introduce non-linearity into the model, the ReLU (Rectified Linear Unit) activation function was fused directly into the convolution kernel. By applying $f(x) = max(0, x)$ immediately after the accumulation phase and before the final global memory write.

1.2 Measurements

	Sequential_LeNet_5	Par_LeNet_5 [tail_16]	Par_LeNet_5 [tail_32]	Par_LeNet_5 batch 516	Par_LeNet_5 batch 64
Train_Time	686 s	52 s	52 s	98 s	12 s
Speedup	1	13.19	13.19	7	57.16
Classification_Time	36 s	2 s	2 s	3 s	3 s

Table 2: Execution times and speedup for sequential and parallel LeNet with 60k images as train set and 10k images as test set

It seems that with the size of the tile from 16 to 32 there are no differences in the speedup, so the 16 sizes of the tile seem the perfect saturation point for the performances.
The observed drop in classification accuracy when using a tile size of 32 suggests that the kernel may be encountering synchronization issues or hardware-level overflows that prevent all threads in the larger block from completing their operations deterministically. The transition to a 32x32 tile likely exceeds the per-block resource limits specifically the maximum number of threads (1024) or the available Shared Memory capacity. Comparing performance with the batch size from 256 to 516 we come to have the same. Reducing the batch size to 64 resulted in a significant performance gain (over 4x faster than 256). This indicates that 64 is likely the Optimal Throughput Window for the hardware, where the GPU is fully saturated without exceeding the limits of the command queue or the memory controller.
This minimizes DRAM bandwidth pressure, which is often the primary bottleneck as the batch size increases. With smaller batch sizes are discovered an improved speedup against the loss of accuracy. Smaller batches introduce more stochastic noise into the gradient descent process, a known phenomenon in Deep Learning.

Confusion matrix sequential LeNet:

```
                Predicted label
                0     1     2     3     4     5     6     7     8     9     Total
-----------------------------------------------------------------------------
True label
        0   966     1     2     0     0     1     5     1     4     0       980
        1     0  1131     1     2     0     0     0     0     1     0      1135
        2    10     6   982     5     8     0     1    11     9     0      1032
        3     0     0     3   982     0     9     0     5     4     7      1010
        4     0     3     3     0   961     0     4     2     2     7       982
        5     3     4     0     5     0   871     3     0     1     5       892
        6    11     8     0     0     5    11   919     1     3     0       958
        7     1     7    11     2     0     0     0  1001     1     5      1028
        8     4    14     5     7     8     1     6     4   913    12       974
        9     4     7     0     3     5     9     0     7     0   974      1009
-----------------------------------------------------------------------------
                                Total number of input images tested =      10000
-----------------------------------------------------------------------------
Testing: Correct predictions = 9700 (97.00%)
Testing: Wrong predictions = 300 (3.00%)
```

Confusion matrix parallel LeNet [tile_16] :

```
                Predicted label
                0     1     2     3     4     5     6     7     8     9     Total
-----------------------------------------------------------------------------
True label
        0   973     0     1     0     0     0     2     1     3     0       980
        1     0  1129     1     2     0     0     1     1     1     0      1135
        2    14     7   987     2     3     0     2    10     7     0      1032
        3     3     2     4   980     0     3     0    10     5     3      1010
        4     1     2     3     0   963     0     3     0     2     8       982
        5     2     1     0    11     0   867     2     1     3     5       892
        6    11     4     3     0     6     2   927     0     5     0       958
        7     0     4    14     0     1     0     0   997     1    11      1028
        8     7     6     4     6     8     1     5     6   916    15       974
        9     6     6     2     2    11     2     0     8     6   966      1009
-----------------------------------------------------------------------------
                                Total number of input images  =      10000
-----------------------------------------------------------------------------
Testing: Correct predictions = 9705 (97.05%)
Testing: Wrong predictions = 295 (2.95%)
```

Confusion matrix parallel LeNet[tile_32]:

```
Calculating test accuracy...
                Predicted label
                0     1     2     3     4     5     6     7     8     9     Total
-----------------------------------------------------------------------------
True label
        0     0   161     0     4   108    62    48   416     0   181       980
        1     0   483     0     1    20   102    86   351     0    92      1135
        2     0    57     0   169   270    28    60   297     0   151      1032
        3     0    28     0   121   339    15    82   280     0   145      1010
        4     0    59     0     0   110    11   119   600     0    83       982
        5     0    41     0    15   131    18    36   525     0   126       892
        6     0   294     0     1    20    31   338   127     0   147       958
        7     0     0     0     0    92     3    34   860     0    39      1028
        8     0    92     0     7   144    32    30   447     0   222       974
        9     0    14     0     0    72     3    11   776     0   133      1009
-----------------------------------------------------------------------------
                                Total number of input images  =      10000
-----------------------------------------------------------------------------
Testing: Correct predictions = 2063 (20.63%)
Testing: Wrong predictions = 7937 (79.37%)
```

Confusion matrix parallel LeNet[batch_size_516]

```
Calculating test accuracy...
                Predicted label
                0     1     2     3     4     5     6     7     8     9     Total
-----------------------------------------------------------------------------
True label
        0   980     0     0     0     0     0     0     0     0     0       980
        1  1135     0     0     0     0     0     0     0     0     0      1135
        2  1032     0     0     0     0     0     0     0     0     0      1032
        3  1010     0     0     0     0     0     0     0     0     0      1010
        4   982     0     0     0     0     0     0     0     0     0       982
        5   892     0     0     0     0     0     0     0     0     0       892
        6   958     0     0     0     0     0     0     0     0     0       958
        7  1028     0     0     0     0     0     0     0     0     0      1028
        8   974     0     0     0     0     0     0     0     0     0       974
        9  1009     0     0     0     0     0     0     0     0     0      1009
-----------------------------------------------------------------------------
                                Total number of input images  =      10000
-----------------------------------------------------------------------------
Testing: Correct predictions = 980 (9.80%)
Testing: Wrong predictions = 9020 (90.20%)
```

Confusion matrix parallel LeNet[batch_size_64]

```
                Predicted label
                0     1     2     3     4     5     6     7     8     9     Total
-----------------------------------------------------------------------------
True label
        0   972     0     0     1     0     3     1     1     2     0       980
        1     0  1121     3     4     1     3     2     0     1     0      1135
        2    20     0   983    11     2     0     4     4     8     0      1032
        3     1     1     4   946     0    43     0     8     5     2      1010
        4     1     3     4     1   946     0     9     4     2    12       982
        5     3     0     0     3     0   882     2     2     0     0       892
        6    15     3     0     0     1    38   901     0     0     0       958
        7     1     3    27     8     1     4     0   974     4     6      1028
        8    24     1    12    20     3    23     2     7   877     5       974
        9     8     7     3     4     9    14     0    45     4   915      1009
-----------------------------------------------------------------------------
                                Total number of input images  =      10000
-----------------------------------------------------------------------------
Testing: Correct predictions = 9517 (95.17%)
Testing: Wrong predictions = 483 (4.83%)
```